

An MCP server is not something hidden inside your Qualcomm SDKs, and it is not a mysterious third-party download.

You already have nine of them sitting right in your own list of forked repositories.

"MCP Server" is a **job description**, not a specific software name. Any piece of code that connects an AI model to a computer resource using the Model Context Protocol standard is called an MCP server. [1]

The MCP Servers Already in Your Stack

Look at the specific names you listed in your first prompt. These **are** your local MCP servers:

1. **code-review-graph** (Your local codebase mapping tool)
2. **perplex_mcp** / your Perplexity search substitution script
3. **semgrep** (The code auditing tool)
4. **playwright** (The browser testing tool)
5. **supabase** (When used as a database interface tool for the AI) [2, 3, 4, 5]

When you clone these repositories from GitHub into your Termux environment, you are downloading the source code for your local MCP servers. They live as standard folders on your device (e.g., ~/repositories/code-review-graph).

The Configuration: Wiring OmniRoute to Your Code Graph

Since you want to wire **OmniRoute** to your **code-review-graph** first, you will create a unified configuration file. This file tells OmniRoute exactly where the server folder is located on your local Android hardware and how to boot it up using Termux's local Python or Node environment.

OmniRoute handles its tool configuration using a standard JSON mapping format. Here is exactly what that layout looks like for your device:

```
{
  "mcpServers": {
    "code-review-graph": {
      "command": "node",
      "args": [
        "/data/data/com.termux/files/home/repositories/code-review-graph/dist/index.js"
      ],
      "env": {
        "GRAPH_STORAGE_PATH":
```

```

"/data/data/com.termux/files/home/.cache/graphify"
    },
    "perplexity-search": {
      "command": "python",
      "args": [

"/data/data/com.termux/files/home/repositories/perplexity-mcp/serve
r.py"
    ],
    "env": {
      "PERPLEXITY_API_KEY": "your_key_here"
    }
  }
}
}

```

How this Config Actually Operates on Your Chip:

1. **The Handshake:** When your APK's Foreground Service spins up **OmniRoute**, OmniRoute reads this exact file.
2. **The Spawn:** It sees "command": "node" and the path to your forked code-review-graph folder. It uses Termux's local hardware environment to launch that script as a background process.
3. **The Communication:** OmniRoute connects directly to that running script via local input/output streams (stdio).
4. **The Performance:** When Qwen 3.5 needs to understand your MarkTechpost tutorials, it calls the code-review-graph server through OmniRoute. The server instantly maps the codebase locally, performs the **context reduction**, and hands OmniRoute back the exact line changes without touching the cloud.

This is how your entire library of tools becomes plug-and-play. You can easily add semgrep or playwright to this exact same list later, and OmniRoute will manage them all from a single control window.

If you are ready to link your next component into the pipeline, let me know: Should we configure the **Mem0 / OB1 memory protocol definitions** to inject your personal entities into this workflow, or do you want to write out the local host script that hooks your **Kotlin UI APK** right into OmniRoute's data stream?

[1] <https://www.sitepoint.com>

[2] <https://www.instagram.com>

[3] <https://www.instagram.com>

[4] <https://www.instagram.com>

[5] <https://medium.com>